

23CSE111 OBJECT ORIENTED PROGRAMMING
S2 B.Tech CSE

Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

CASE STUDY

PHASE – 2 (Design)
Architectural Modelling (CO2)

Group No: C5

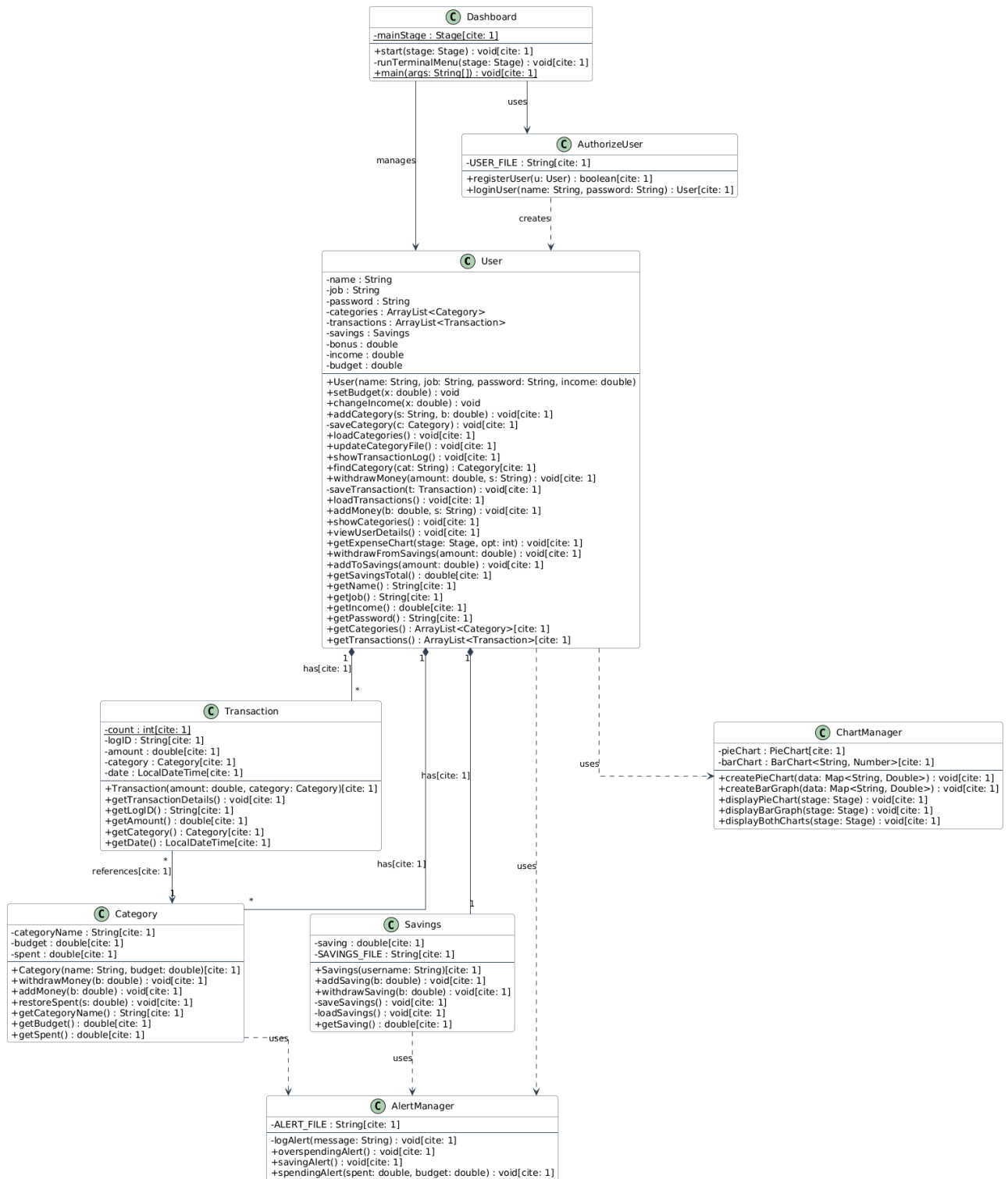
Team Members :

Roll No	Student Name
AM.SC.U4CSE25218	Gorrela Tulasi Lasya
AM.SC.U4CSE25220	Hansika L Chawla
AM.SC.U4CSE25230	Lekshmi Nandana
AM.SC.U4CSE25236	Nandana Jayakumar

1. Problem Title: Personal Finance Manager

STRUCTURAL DESIGN

2. Class diagram (*draw class diagram that shows different classes and their relationships. Use UML notations*)



BEHAVIUIORAL DESIGN

3. Use-case diagram

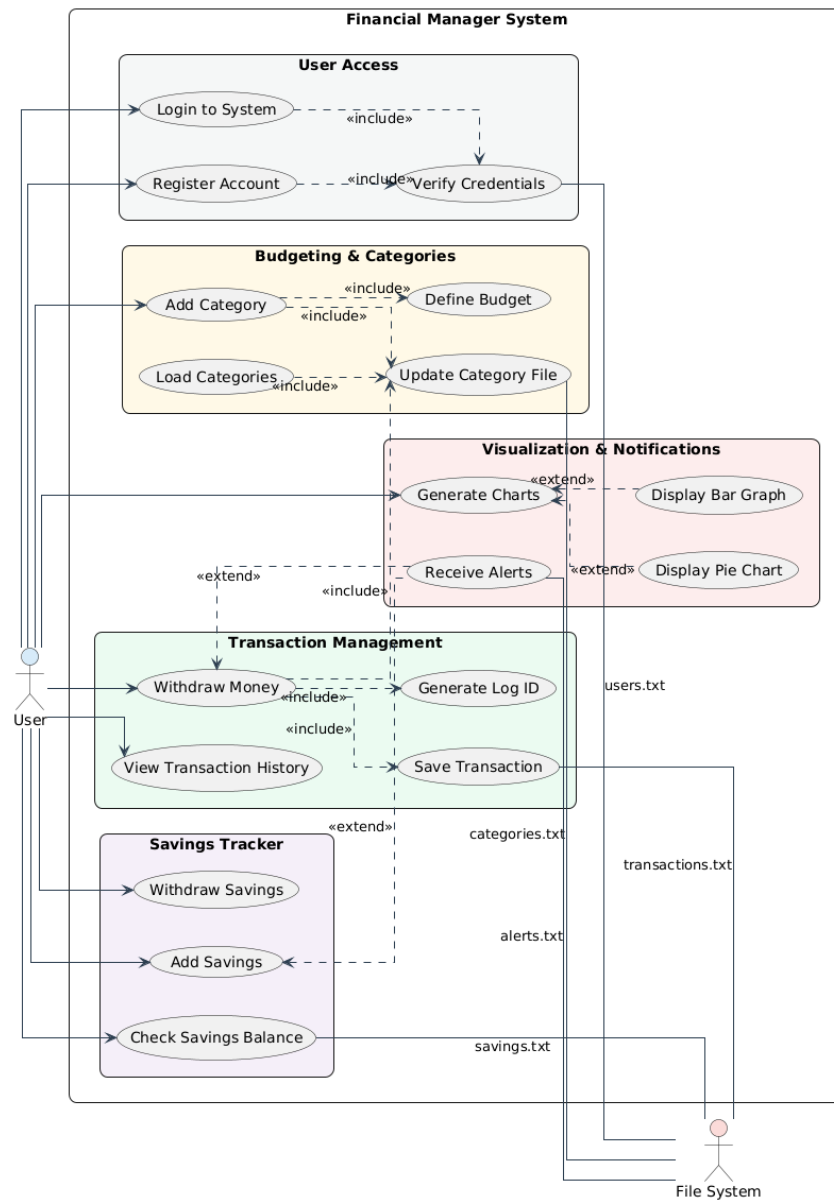
a. List of actors

1. Users
2. System

b. List of use-cases

- **Withdraw Money** – Allows the user to deduct a specified amount from their account balance.
- **Add Money** – Allows the user to deposit or credit funds into their account.
- **View Transaction Log** – Displays a history of all past financial transactions made by the user.
- **Authenticate** – Verifies the identity of the user before granting access to the system.
- **Login** – Allows a registered user to sign in using their credentials.
- **Register Account** – Enables a new user to create an account in the system.
- **Delete Account** – Allows the user to permanently remove their account from the system.
- **Check Budget Status** – Shows the user how much of their set budget has been used or remains.
- **Set Budget** – Allows the user to define a spending limit for a specific period or category.
- **Add Category** – Enables the user to create a new spending or savings category.
- **View Categories** – Displays all existing categories created by the user.
- **Overspending Alert** – Automatically notifies the user when their spending exceeds the set budget.
- **Spending Alert** – Sends a warning when the user's spending reaches a defined threshold.
- **Savings Alert** – Notifies the user when their savings reach or fall below a target amount.
- **View Dashboard** – Provides the user with an overview of their overall financial status.
- **View Pie Chart** – Displays a visual breakdown of spending across different categories.
- **View Bar Graph** – Shows a comparative graphical view of income and expenses over time.
- **View User Details** – Displays the personal and account information of the logged-in user.
- **Withdraw from Savings** – Allows the user to transfer funds out of their savings pool.
- **Add to Savings** – Enables the user to move funds into their savings for future use.

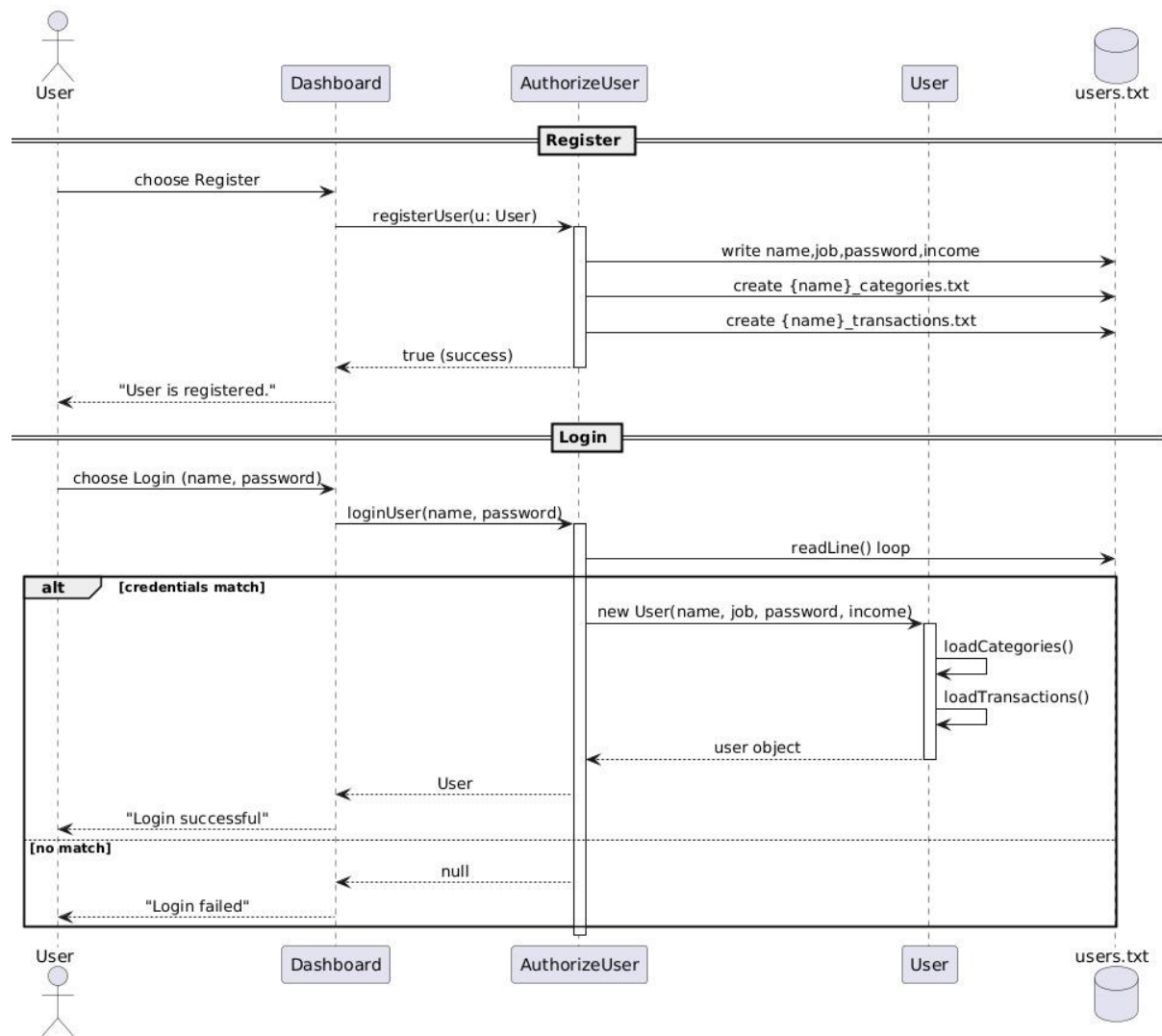
c. Use case diagram (use UML notations)



4. Sequence diagram (use UML notations)

SCENARIO:

When user creates account and Logins to their account then our application follows the sequence as shown below:

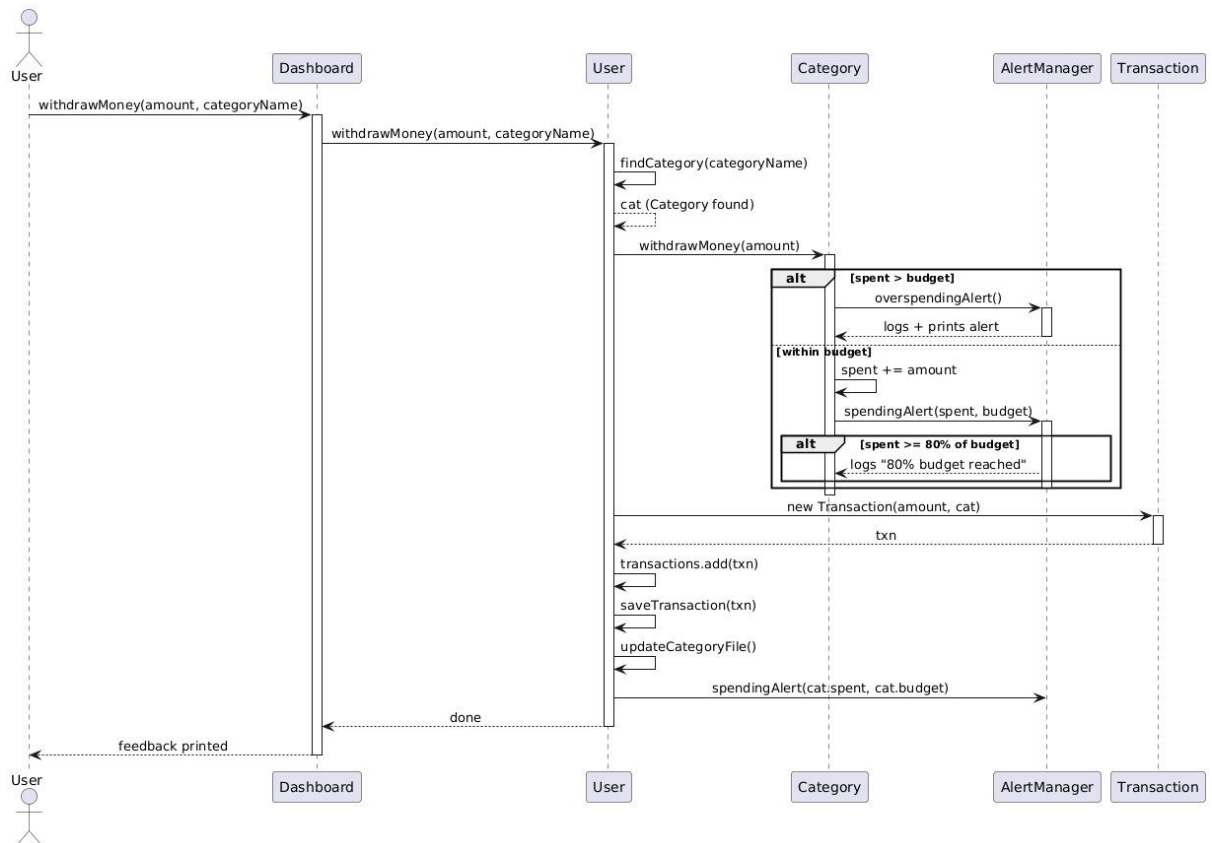


e Diagram Explanation:

The **Registration** process functions as a file-initialization sequence where the Dashboard sends user details to the AuthorizeUser class. The system then executes a `write` operation to `users.txt` to store the credentials and immediately performs a `createNewFile()` operation to generate unique `_categories.txt` and `_transactions.txt` files tied to that specific username. During the **Login** phase, the system runs a `readLine()` loop through `users.txt` to perform string matching on the provided name and password. Upon a successful match, a `User` object is instantiated in memory, which automatically invokes `loadCategories()` and `loadTransactions()`. These methods parse the user's local text files and reconstruct the `ArrayList` data structures needed for the active session.

SCENARIO:

When User wants to withdraw money:



Sequence Diagram Explanation:

The **Withdrawal** process begins with the User object executing a `findCategory()` loop to locate the target Category object within the active ArrayList. Once located, the

`withdrawMoney(amount)` method is called, which increments the `spent` variable and triggers a logic check within the `AlertManager`. The system evaluates the condition `spent >= 0.8 * budget`; if true, it executes the `logAlert()` method to write a warning to `alerts.txt` and print it to the console. Simultaneously, the `User` class instantiates a new `Transaction` object, which auto-generates a unique `logID` using a static counter. The process concludes with two persistent storage updates: `saveTransaction()` appends the new entry to the transaction file, and `updateCategoryFile()` performs a full overwrite of the category file to sync the new balance.